SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	**192511053**
Name	**B Santhosh Kumar**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

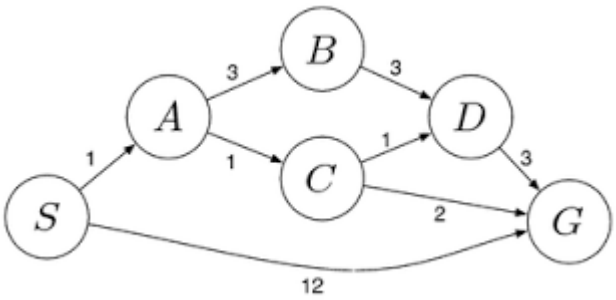
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path

from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I __B Santhosh Kumar/192511053_ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar
Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question:1- Solve the Water Jug Problem using Artificial Intelligence Search

Aim:
To solve the Water Jug Problem using Artificial Intelligence search techniques and obtain **exactly 2 gallons of water in the 4-gallon jug** using the given constraints.

Introduction:
The **Water Jug Problem** is one of the classical Artificial Intelligence (AI) search problems. It is commonly used to demonstrate **state-space representation**, **problem formulation**, and **search algorithms** such as **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.
In this problem, there are two unmarked jugs:

- **Jug A = 4 gallons**
- **Jug B = 3 gallons**

There is no measuring scale on either jug. The objective is to obtain **exactly 2 gallons of water in the 4-gallon jug** by performing a sequence of legal operations.

Problem Statement:
You are given two jugs:

- One **4-gallon jug**
- One **3-gallon jug**

Both jugs are initially empty.
A water pump is available that can completely fill either jug.
The following operations are allowed:

- Fill a jug completely.
- Empty a jug completely onto the ground.
- Pour water from one jug into the other until either:
  - the source jug becomes empty, or
  - the destination jug becomes full.

No measuring devices are available.
The goal is to obtain **exactly 2 gallons of water in the 4-gallon jug**.

Assumptions:
The following assumptions are considered:

1. Both jugs are initially empty.
2. Unlimited water is available through the pump.
3. Water may be discarded onto the ground.
4. No markings exist on either jug.
5. Water can be transferred between the two jugs.
6. Water transfer stops only when one jug becomes full or the other becomes empty.
7. No other measuring equipment is available.

State Space Representation:

A state is represented as:

$$(A, B)$$

Where

- **A = Amount of water in the 4-gallon jug**
- **B = Amount of water in the 3-gallon jug**

Example:

State	Meaning
(0,0)	Both jugs empty
(4,0)	4-gallon jug full
(4,3)	Both jugs full
(2,3)	Goal nearly reached

Initial State

$$(0, 0)$$

Both jugs are empty.

Goal State:

$$(2, x)$$

where **x can be any value (0–3)**.
In this solution,
Goal State:

$$(2, 0)$$

Operators (Possible Actions)

The available operators are:

1. Fill 4-gallon jug.
2. Fill 3-gallon jug.
3. Empty 4-gallon jug.
4. Empty 3-gallon jug.
5. Pour 4-gallon jug into 3-gallon jug.
6. Pour 3-gallon jug into 4-gallon jug.

State Space Diagram:

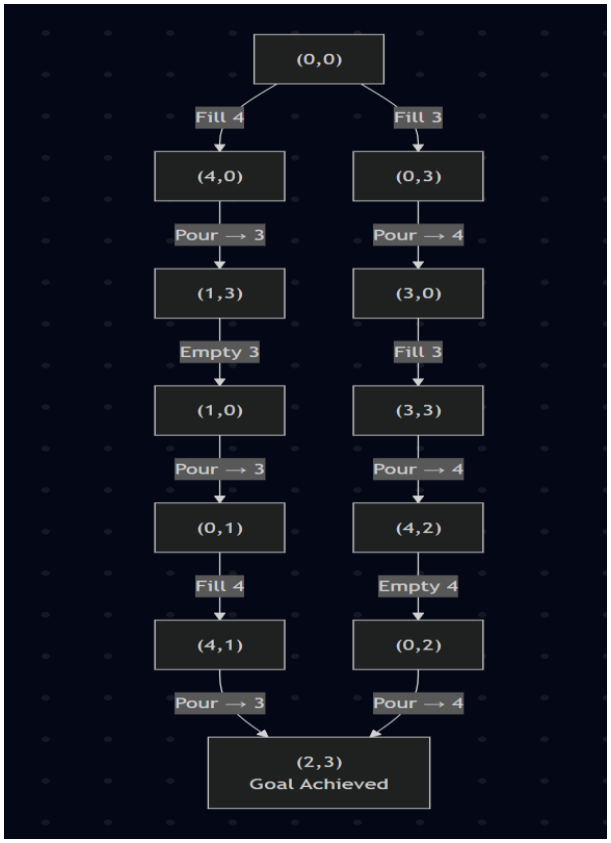


Figure 1: State Space Representation of Water Jug Problem

Step-by-Step Solution:

Step	Operation	State (4-gallon,3-gallon)
1	Initial State	(0,0)
2	Fill 3-gallon jug	(0,3)
3	Pour into 4-gallon jug	(3,0)
4	Fill 3-gallon jug again	(3,3)
5	Pour into 4-gallon jug until full	(4,2)
6	Empty 4-gallon jug	(0,2)
7	Pour remaining 2 gallons into 4-gallon jug	(2,0)

Solution Explanation

Step 1

Initially,
Both jugs are empty.
4-Gallon Jug = 0

3-Gallon Jug = 0
State
(0,0)

Step 2

Fill the 3-gallon jug completely.
4-Gallon = 0

3-Gallon = 3
State
(0,3)

Step 3
Pour water from the 3-gallon jug into the 4-gallon jug.
4-Gallon = 3

3-Gallon = 0
State
(3,0)

Step 4
Fill the 3-gallon jug again.
State
(3,3)

Step 5
Pour water from the 3-gallon jug into the 4-gallon jug.
The 4-gallon jug already contains 3 gallons.
Only **1 gallon** can be added.
Therefore,
4-Gallon = 4

3-Gallon = 2
State
(4,2)

Step 6
Empty the 4-gallon jug.
State
(0,2)

Step 7
Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.
Now,
4-Gallon = 2

3-Gallon = 0
State
(2,0)

Goal Achieved:
Exactly **2 gallons** are present in the **4-gallon jug**.

State Transition Table

Current State	Action	Next State
(0,0)	Fill 3-gallon jug	(0,3)
(0,3)	Pour into 4-gallon jug	(3,0)
(3,0)	Fill 3-gallon jug	(3,3)
(3,3)	Pour into 4-gallon jug	(4,2)
(4,2)	Empty 4-gallon jug	(0,2)
(0,2)	Pour into 4-gallon jug	(2,0)

Search Technique Used

The Water Jug Problem is generally solved using **State Space Search**.
The search techniques include:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search
- Iterative Deepening Search

Among these, **Breadth-First Search (BFS)** is commonly preferred because it guarantees the shortest sequence of operations to reach the goal state.

Applications of Water Jug Problem:

- Artificial Intelligence problem solving
- Robotics
- Automated planning
- State-space search
- Game development
- Constraint satisfaction problems
- Algorithm design
- Educational AI demonstrations

Advantages

- Demonstrates state-space representation effectively.
- Simple example for learning AI search algorithms.
- Helps understand BFS and DFS concepts.
- Useful for teaching planning and reasoning techniques.
- Can be extended to more complex search problems.

Limitations

- Applicable only to small finite state spaces.
- Does not represent real-world uncertainty.
- State space grows rapidly with more jugs.
- Manual solution becomes difficult for larger capacities.

Conclusion

The Water Jug Problem is a classical Artificial Intelligence problem that demonstrates the concepts of **state-space representation**, **problem formulation**, and **search strategies**. By applying valid operations such as filling, emptying, and transferring water between jugs, the goal state **(2,0)** is successfully reached, meaning exactly **2 gallons of water** are obtained in the **4-gallon jug**. This problem illustrates how AI agents systematically explore possible states to find a valid and optimal solution.

Question 2: Study of Mars Rover as an Intelligent Agent

Aim:

To study the Mars Rover as an intelligent autonomous agent and analyze its percepts, operating environment, actions, performance measures, and suitable agent architecture used for exploring and analyzing the surface of Mars.

Introduction:

A Mars Rover is an intelligent robotic vehicle developed by NASA to explore the surface of Mars. It is equipped with advanced sensors, cameras, robotic arms, scientific instruments, and onboard computers that enable it to perform scientific experiments without continuous human control.

Since Mars is approximately **225 million kilometers away from Earth,** communication signals require several minutes to travel between Earth and Mars. Because of this delay, the rover cannot depend entirely on instructions from scientists on Earth. Instead, it uses Artificial Intelligence (AI) to perceive its surroundings, make decisions, avoid obstacles, navigate safely, collect soil and rock samples, analyze them, and transmit the collected data back to Earth.

The Mars Rover is considered an Intelligent Agent because it continuously senses the environment, processes information, makes decisions, and performs appropriate actions to achieve mission objectives.

Some well-known Mars Rovers include:

- Sojourner (1997)
- Spirit (2004)
- Opportunity (2004)
- Curiosity (2012)
- Perseverance (2021)

These missions have significantly contributed to the scientific understanding of Mars by studying its geology, atmosphere, climate, and the possibility of ancient life.

Figure 1: Mars Rover exploring the Martian surface.

Problem Statement:

The Mars Rover functions as an intelligent autonomous agent that explores the unknown environment of Mars. It must collect scientific data, identify rocks and minerals, avoid obstacles, navigate safely across rough terrain, and communicate important findings back to Earth with minimal human intervention.

The objective is to analyze the Mars Rover by answering the following AI concepts:

1. What are the percepts for this agent?
2. Characterize the operating environment.
3. What actions can the agent perform?
4. How can the performance of the agent be evaluated?
5. Which agent architecture is most suitable and why?

Overview of Mars Rover as an Intelligent Agent:

Feature	Description
Agent Name	Mars Rover
Agent Type	Intelligent Autonomous Agent
Developer	NASA
Operating Environment	Surface of Mars
Primary Goal	Explore Mars and collect scientific information
Sensors	Cameras, Spectrometer, Temperature Sensor, IMU
Actuators	Wheels, Robotic Arm, Drill, Camera Mast
Communication	High-Gain Antenna to Earth

Working Principle of Mars Rover:

The Mars Rover follows a continuous cycle of sensing, decision-making, acting, and learning.

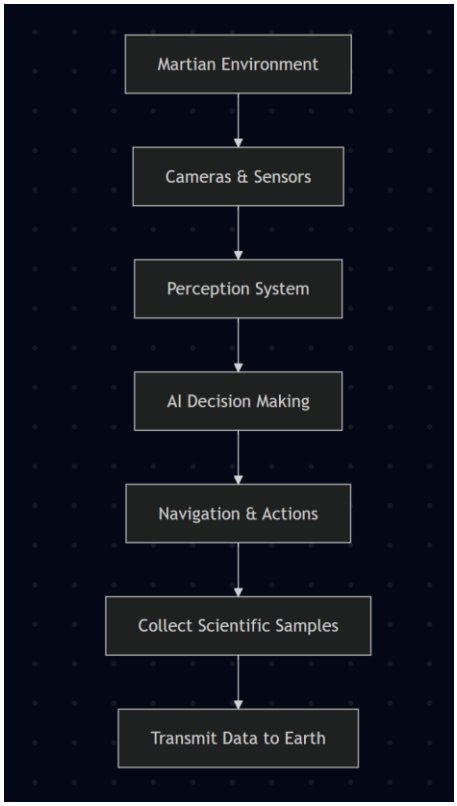


Figure 2: Basic Working Cycle of the Mars Rover.

i What are the Percepts for this Agent?

Definition
A Percept is any information received by an intelligent agent through its sensors. The Mars Rover continuously collects information from its surroundings to understand the environment and make appropriate decisions.

Major Percepts of the Mars Rover

- Images of rocks and mountains
- Surface texture and soil composition
- Distance to nearby obstacles
- Temperature of the atmosphere
- Atmospheric pressure
- Wind speed and direction
- Radiation levels
- Dust concentration
- Wheel position and movement
- Battery level
- Current location and orientation

These percepts help the rover determine safe paths, identify scientific targets, and monitor its own health.

Sensors and Their Percepts:

Sensor	Percept Collected	Purpose
Navigation Camera (NavCam)	Terrain images	Safe navigation
Mast Camera (MastCam)	High-resolution images	Scientific observation
Hazard Camera (HazCam)	Obstacles and slopes	Collision avoidance
Spectrometer	Mineral composition	Rock analysis
Temperature Sensor	Atmospheric temperature	Weather monitoring
Pressure Sensor	Air pressure	Environmental analysis
Radiation Sensor	Radiation level	Safety assessment
Wheel Encoder	Wheel rotation	Distance measurement
IMU (Inertial Measurement Unit)	Position and orientation	Navigation

Perception Process

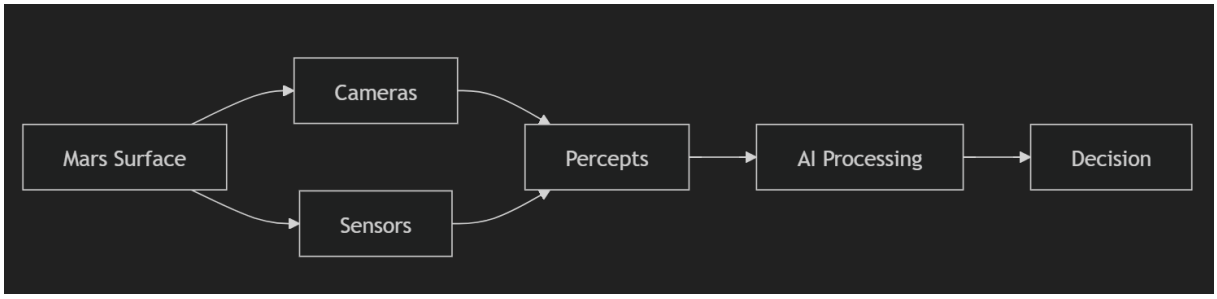


Figure 3: Perception Process of the Mars Rover.

ii. Characterize the Operating Environment

The operating environment describes the conditions under which the Mars Rover performs its tasks.
The Martian environment is harsh, unpredictable, and constantly changing. Therefore, the rover must operate autonomously using Artificial Intelligence.

Characteristics of the Operating Environment

Property	Type	Explanation
Observability	Partially Observable	The rover cannot observe the complete environment at once.
Deterministic	Stochastic	Environmental changes such as dust storms make outcomes uncertain.
Episodic / Sequential	Sequential	Each action affects future actions.
Static / Dynamic	Dynamic	Weather and terrain conditions change continuously.
Discrete / Continuous	Continuous	Movement, temperature, and sensor values change continuously.
Single / Multi-Agent	Single Agent	Only one rover performs the assigned mission.
Known / Unknown	Unknown	The rover explores areas that have never been visited before.

Explanation of Environment Characteristics

1.Partially Observable

The Mars Rover cannot view the entire Martian surface simultaneously. It relies on cameras and sensors to observe only the nearby environment.

2. Stochastic (Uncertain)

Unexpected events such as dust storms, loose rocks, steep slopes, and communication delays make the environment unpredictable.

3. Sequential

Every action performed by the rover influences the next action. For example, selecting an unsafe path may consume extra battery power and delay the mission.

4. Dynamic

Environmental conditions continuously change due to weather, dust, sunlight, and temperature variations.

5. Continuous

Movement, wheel rotation, speed, and sensor readings change continuously rather than in fixed steps.

6. Single Agent

The Mars Rover independently performs exploration tasks without assistance from other robots.

7. Unknown Environment

The rover explores unexplored regions of Mars where no prior information is available.

Operating Environment:

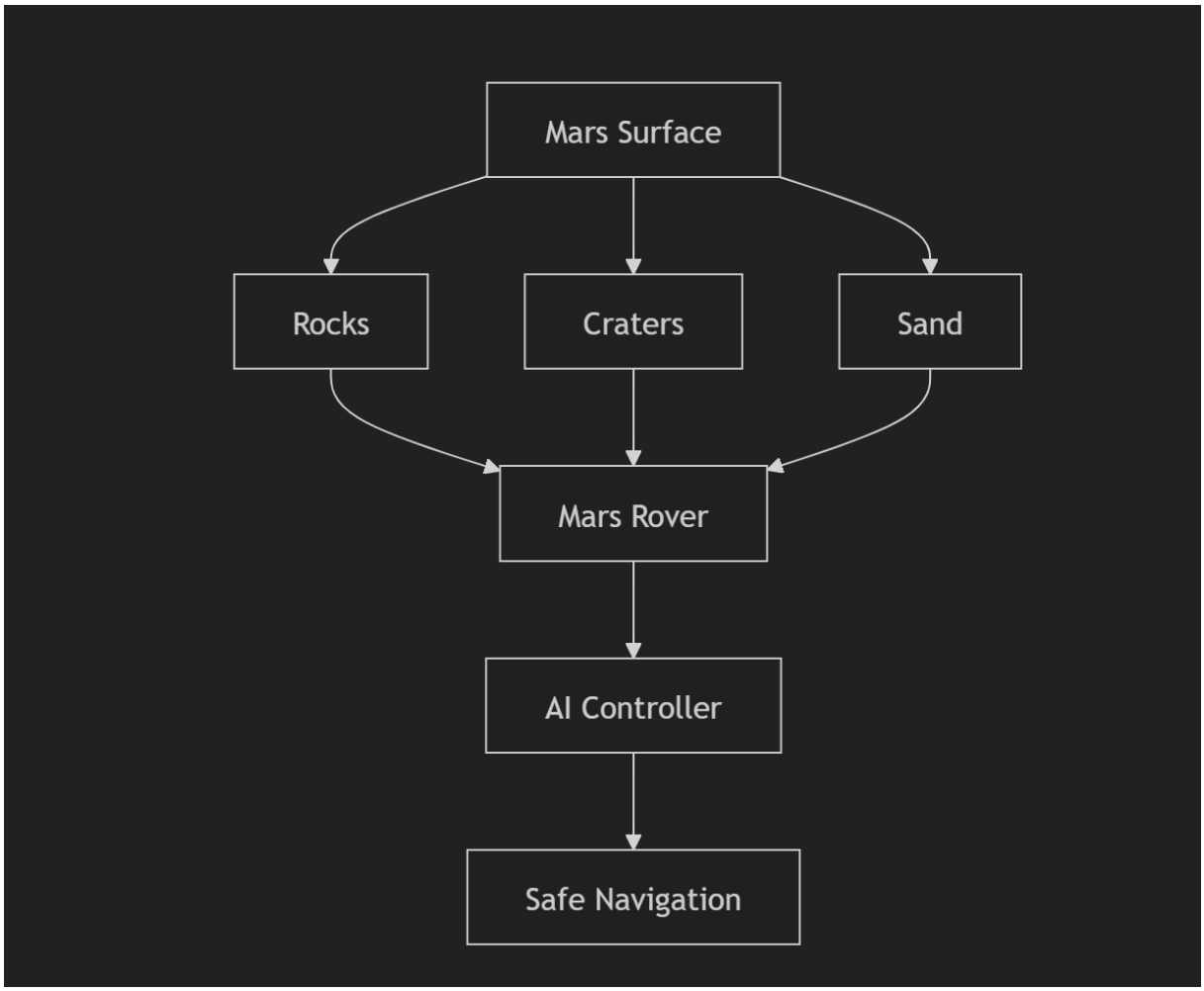


Figure 4: Operating Environment of the Mars Rover.

iii. What are the Actions the Agent Can Take?

The Mars Rover performs various actions based on the information collected from its sensors. These actions help it accomplish its scientific mission while ensuring safe navigation on the Martian surface.

Major Actions of the Mars Rover

1. **Navigation Actions**
    - Move Forward
    - Move Backward
    - Turn Left
    - Turn Right
    - Stop
2. **Scientific Actions**
    - Capture high-resolution images
    - Collect rock and soil samples
    - Drill into rocks
    - Analyze minerals and chemical composition
    - Measure atmospheric conditions
3. **Communication Actions**
    - Transmit scientific data to Earth
    - Receive mission updates from Earth
    - Send rover health status
4. **Safety Actions**

- Detect obstacles
- Avoid collisions
- Monitor battery level
- Enter safe mode during emergencies

Table 3: Actions Performed by the Mars Rover

Action Type	Activities	Purpose
Navigation	Move, Turn, Stop	Safe movement across Mars
Scientific	Sample collection, Rock drilling	Scientific research
Communication	Send and receive data	Mission control
Safety	Obstacle avoidance, Battery monitoring	Prevent accidents
Environmental	Measure temperature, pressure	Weather analysis

Action Cycle:

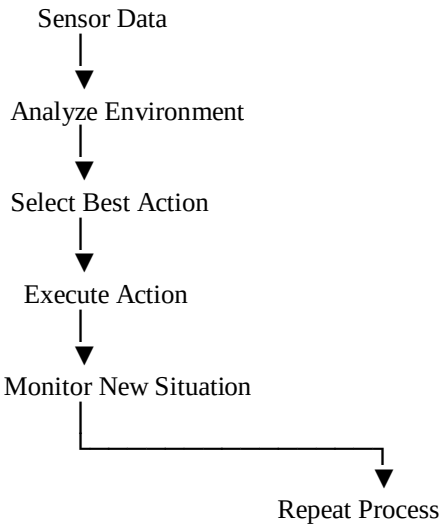


Figure 5: Action cycle of the Mars Rover.

iv. How Can One Evaluate the Performance of the Agent?

The performance of the Mars Rover is evaluated based on how efficiently and accurately it completes its assigned mission. A good intelligent agent should achieve its objectives while minimizing errors, energy consumption, and risks.

Performance Measures

- Accuracy of navigation
- Successful obstacle avoidance
- Number of scientific samples collected
- Accuracy of scientific analysis
- Battery efficiency
- Communication reliability
- Distance covered safely
- Completion of mission objectives

Table 4: Performance Evaluation Parameters

Performance Measure	Description
Navigation Accuracy	Ability to move safely without collisions
Obstacle Avoidance	Detect and avoid rocks, cliffs, and craters
Sample Collection	Collect correct rock and soil samples
Scientific Analysis	Produce accurate experimental results

Communication	Successfully transmit data to Earth
Battery Management	Efficient use of available power
Reliability	Operate continuously without failures
Mission Completion	Achieve all planned objectives

Performance Evaluation Criteria:

A Mars Rover is considered successful if it:

- Safely navigates the Martian surface.
- Collects valuable scientific samples.
- Sends accurate information to Earth.
- Avoids obstacles and hazardous terrain.
- Uses battery power efficiently.
- Completes the mission within the planned duration.

v. What Sort of Agent Architecture is Most Suitable? Why?

The most suitable architecture for the Mars Rover is the Model-Based Goal-Based Agent Architecture.

This architecture combines an internal model of the environment with goal-oriented planning, allowing the rover to make intelligent decisions even when complete information is unavailable.

Why is this Architecture Suitable?

- Maintains an internal representation of the environment.
- Stores previous observations and experiences.
- Plans future movements based on mission goals.
- Makes autonomous decisions.
- Adapts to unknown and changing environments.
- Handles communication delays effectively.
- Selects the safest and most efficient path.

Model-Based Goal-Based Agent Architecture:

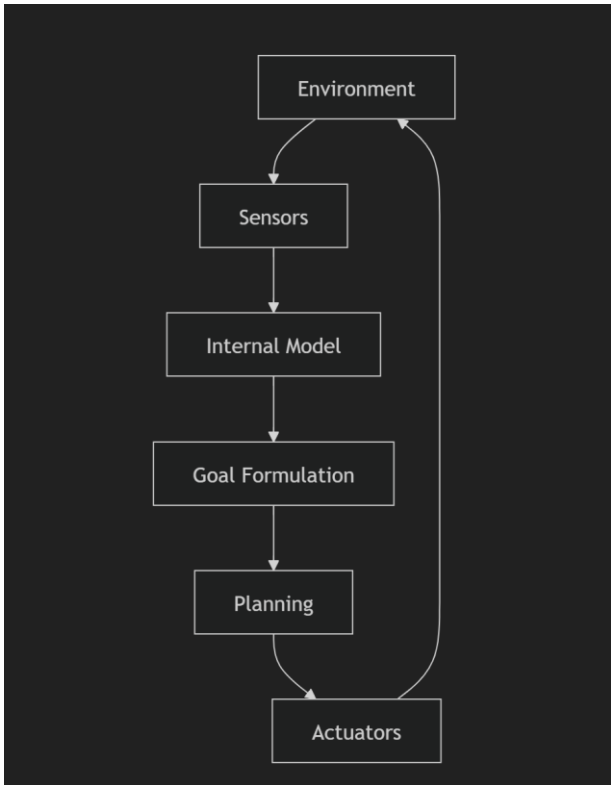


Figure 6: Model-Based Goal-Based Agent Architecture used by the Mars Rover.

Why Not Other Agent Types?

Agent Type	Suitable?	Reason
Simple Reflex Agent	✗ No	Cannot handle unknown environments.
Model-Based Reflex Agent	✓ Partially	Maintains an internal state but lacks goal planning.
Goal-Based Agent	✓ Yes	Selects actions to achieve mission goals.
Utility-Based Agent	✓ Good	Chooses actions with maximum efficiency.
Model-Based Goal-Based Agent	✓ Best Choice	Combines internal modeling with goal-oriented planning, making it ideal for Mars exploration.

Advantages of the Mars Rover

- Operates autonomously with minimal human intervention.
- Explores dangerous and inaccessible environments.
- Performs accurate scientific experiments.
- Avoids obstacles using intelligent navigation.
- Provides valuable data for planetary research.
- Improves the efficiency and safety of space missions.

Applications

- Space exploration
- Planetary science research
- Geological surveys
- Environmental monitoring
- Autonomous robotics
- Artificial Intelligence research
- Remote exploration of hazardous environments
- Future human Mars missions

Conclusion

The Mars Rover is an excellent example of an intelligent autonomous agent that applies Artificial Intelligence to perform complex exploration tasks on Mars. It continuously perceives its surroundings using sensors, processes information through onboard AI systems, plans safe navigation routes, collects and analyzes scientific samples, and communicates valuable data to Earth. Since it operates in a partially **observable, dynamic, sequential, and** uncertain environment, the Model-Based Goal-Based Agent Architecture is the most appropriate choice. This architecture enables the rover to maintain an internal model of its environment, make informed decisions, adapt to changing conditions, and successfully accomplish its mission objectives. Thus, the Mars Rover demonstrates the practical application of AI in real-world space exploration and autonomous robotic systems.

Question 3: Explore the Search Space Using Search Strategies and Formulate the Problem Components for the 8-Queens Problem

Aim:

To formulate the **8-Queens Problem** using Artificial Intelligence search strategies and identify its problem components, state-space representation, search techniques, and solution methodology.

Introduction:

The **8-Queens Problem** is one of the classical problems in Artificial Intelligence (AI) and Constraint Satisfaction Problems (CSP). The objective is to place **eight queens** on an **8 × 8 chessboard** such that no queen attacks another queen.

In the game of chess, a queen can move:

- Horizontally (rows)
- Vertically (columns)
- Diagonally

Therefore, no two queens should be placed in the same row, same column, or same diagonal.

The 8-Queens Problem is widely used to demonstrate **state-space search**, **backtracking algorithms**, and **constraint satisfaction techniques** in Artificial Intelligence.

Problem Statement:

Place **8 queens** on an **8 × 8 chessboard** such that:

- No two queens are placed in the same row.
- No two queens are placed in the same column.
- No two queens are placed in the same diagonal.

The objective is to formulate the problem using AI search strategies and determine a valid solution.

8-Queens Chessboard:

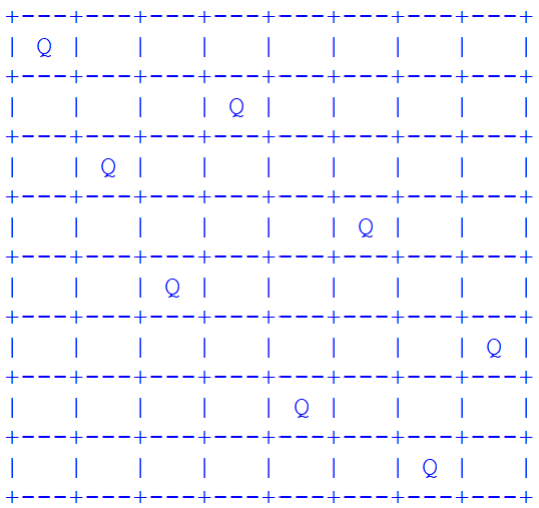


Figure 1: Example of an 8-Queens solution.

State Space Representation:

A **state** represents a partial or complete arrangement of queens on the chessboard.

Each state can be represented as:

(Q1, Q2, Q3, …, Q8)

where each value indicates the row position of the queen in its respective column.

Example:

(1,5,8,6,3,7,2,4)
This means:

Column	Queen Position
1	Row 1
2	Row 5
3	Row 8
4	Row 6
5	Row 3
6	Row 7
7	Row 2
8	Row 4

Problem Formulation:

The 8-Queens problem can be formulated using the following AI components.

Component	Description
Initial State	Empty chessboard
State Space	All possible queen arrangements
Successor Function	Place one queen in the next column
Goal Test	Eight queens placed without attacking each other
Path Cost	Uniform (each move has equal cost)

Initial State:

Initially, no queens are placed on the board.

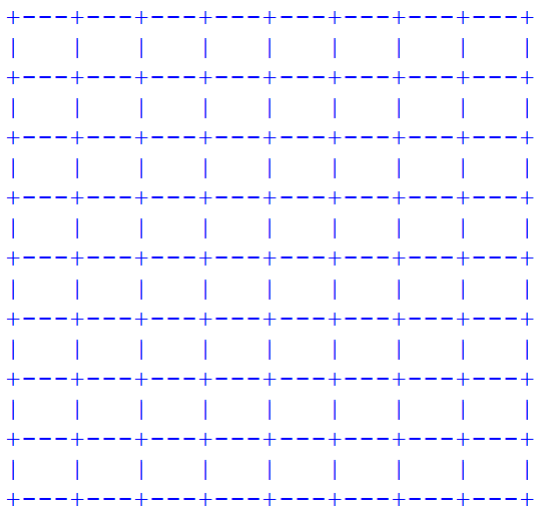


Figure 2: Initial State

Goal State

A valid arrangement where:

- No queens share the same row.
- No queens share the same column.
- No queens share the same diagonal.

Example Goal State:

(1,5,8,6,3,7,2,4)

Successor Function

The successor function generates new states by placing one queen in the next column while ensuring that it does not attack any previously placed queen.

For every column:

- Try placing a queen in each row.
- Check for row and diagonal conflicts.
- Accept only valid positions.

Goal Test

A state is considered a goal state if:

- Exactly eight queens are placed.
- No two queens attack each other.
- All constraints are satisfied.

Search Strategy:

Several search strategies can solve the 8-Queens problem.

Search Strategy	**Description**
Breadth-First Search (BFS)	Explores all states level by level.
Depth-First Search (DFS)	Explores one branch completely before backtracking.
Backtracking Search	Most efficient; removes invalid choices immediately.
Hill Climbing	Improves solution iteratively.
Genetic Algorithm	Uses evolutionary optimization for large variants.

Backtracking Search:

Backtracking is the most commonly used algorithm because it:

- Places queens one by one.
- Checks constraints after each placement.
- Removes invalid placements immediately.
- Continues until a valid solution is found.

State Space Search Diagram:

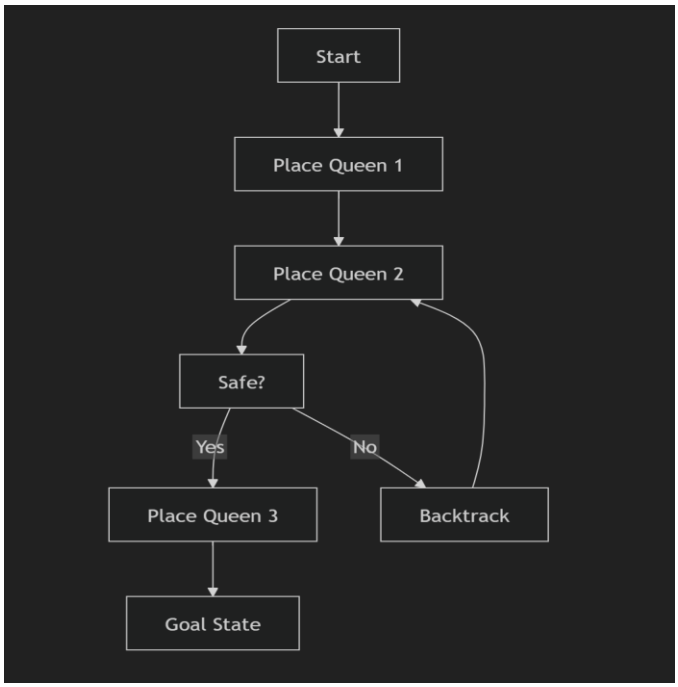


Figure 3: State Space Search using Backtracking.

Algorithm (Backtracking):

Step 1: Start with an empty chessboard.
Step 2: Place a queen in the first column.
Step 3: Move to the next column.
Step 4: Check whether the queen is safe.

- Same row
- Same diagonal

Step 5: If safe, place the queen.
Step 6: If not safe, try the next row.
Step 7: If no row is valid, backtrack to the previous column.
Step 8: Repeat until all eight queens are placed.

Solution Example:

Queen	Position (Row, Column)
Q1	(1,1)
Q2	(5,2)
Q3	(8,3)
Q4	(6,4)
Q5	(3,5)
Q6	(7,6)
Q7	(2,7)
Q8	(4,8)

This arrangement satisfies all constraints.

Applications

- Artificial Intelligence
- Constraint Satisfaction Problems (CSP)
- Scheduling Problems
- Puzzle Solving
- Robotics
- Resource Allocation
- Search Algorithms
- Optimization Problems

Advantages:

- Demonstrates state-space search effectively.
- Introduces constraint satisfaction concepts.
- Efficiently solved using backtracking.
- Improves logical reasoning skills.
- Forms the basis for many AI optimization problems.

Limitations:

- Large search space (8^8 possible arrangements before pruning).
- Brute-force methods are computationally expensive.
- Requires efficient pruning techniques for optimal performance.

Conclusion:

The **8-Queens Problem** is a classic Artificial Intelligence problem that demonstrates **state-space search** and **constraint satisfaction** techniques. By placing one queen at a time and eliminating invalid arrangements using **Backtracking Search**, the search space is significantly reduced, leading to an efficient solution.

Question 4: OLA Cab Booking as a Problem-Solving Agent

Aim:

To identify the **Artificial Intelligence problem-solving agent** used in the OLA Cab Booking application and develop the pseudocode for booking a cab from one location to another.

Introduction:

Artificial Intelligence (AI) enables software applications to make intelligent decisions based on user inputs and environmental conditions. One of the practical applications of AI is the **OLA Cab Booking System**, which helps customers travel from one location to another by selecting the most suitable cab based on their preferences.

The OLA mobile application acts as an **intelligent problem-solving agent** by receiving the customer's request, processing available information such as pickup location, destination, available cab types, estimated fare, traffic conditions, and driver availability, and then recommending the most appropriate cab. It also determines the optimal route, assigns a nearby driver, and continuously tracks the trip until the destination is reached.

Problem Statement;

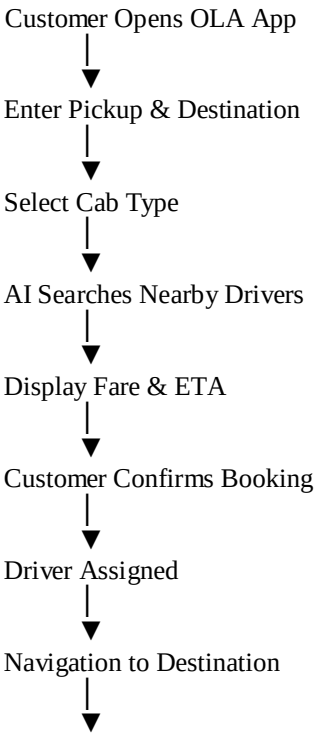
A customer wants to travel from one location to another using the **OLA Cab Booking mobile application**. The customer can select any cab type such as **Mini, Micro, Sedan, Prime, Shared**, etc., according to comfort, travel requirements, and budget.

The AI agent must:

- Receive the customer's pickup and destination locations.
- Display the available cab types.
- Estimate fare and travel time.
- Allocate the nearest available driver.
- Navigate using the best route.
- Complete the trip safely and efficiently.

The task is to identify the **type of problem-solving agent** used in this scenario and write the pseudocode for the booking process.

OLA Cab Booking Workflow



Payment & Trip Completion

Figure 1: Workflow of the OLA Cab Booking System.

Problem-Solving Agent:

- The **Goal-Based Problem-Solving Agent** is the most suitable AI agent for the OLA Cab Booking application.
- A Goal-Based Agent selects actions that help achieve a specific goal. In this case, the goal is to **transport the customer safely from the pickup location to the destination while minimizing travel time and cost**.
- The agent considers various factors such as customer preferences, cab availability, traffic conditions, shortest routes, and estimated arrival time before making a decision.

Why Goal-Based Agent?

A Goal-Based Agent is suitable because it:

- Selects the best cab according to the customer's requirements.
- Finds the shortest or fastest route.
- Assigns the nearest available driver.
- Responds to changing traffic conditions.
- Reaches the destination efficiently.
- Continuously monitors the trip until completion.

Characteristics of the OLA AI Agent:

Feature	Description
Agent Type	Goal-Based Problem-Solving Agent
Goal	Transport customer safely to destination
Environment	Dynamic and Real-Time
Sensors	GPS, Maps, Traffic Data, Customer Input
Actuators	Driver Assignment, Route Selection, Notifications
Performance Measure	Fast, Safe, and Cost-Effective Trip

PEAS Representation:

PEAS stands for **Performance Measure, Environment, Actuators, and Sensors**, which describes the behavior of an intelligent agent.

Component	Description
Performance Measure	Minimum waiting time, shortest route, customer satisfaction, safe journey, accurate fare estimation
Environment	Roads, traffic, GPS location, customers, drivers, weather conditions
Actuators	Driver assignment, route selection, notifications, trip updates, payment confirmation
Sensors	GPS, customer location, destination, traffic information, driver availability

Working of the OLA Problem-Solving Agent:

The AI agent performs the following sequence of operations:

1. Customer opens the OLA application.
2. The customer enters the pickup location.
3. The customer enters the destination.
4. The application displays available cab types.
5. The customer selects a preferred cab.

6. The AI searches for nearby drivers.
7. The estimated fare and arrival time are calculated.
8. The customer confirms the booking.
9. The nearest driver is assigned.
10. GPS navigation guides the driver to the destination.
11. The trip is completed and payment is processed.

AI Decision Process:

Figure 2: Decision-making process of the OLA AI Agent.

Algorithm:

Step 1: Open the OLA application.
Step 2: Enter the pickup location.
Step 3: Enter the destination.
Step 4: Display available cab types.
Step 5: Customer selects the preferred cab.
Step 6: Search for nearby available drivers.
Step 7: Calculate estimated fare and travel time.
Step 8: Assign the nearest available driver.
Step 9: Start navigation using GPS.
Step 10: Complete the trip.
Step 11: Generate the bill and process payment.

Pseudocode:

BEGIN

Start OLA Application

```
Input Pickup_Location
Input Destination

Display Available_Cab_Types

Customer_Selects_Cab

Search Nearby_Drivers

IF Driver_Available THEN

    Calculate Fare

    Calculate Estimated_Time

    Assign Nearest_Driver

    Display Driver Details

    Start GPS Navigation

    Reach Destination

    Generate Bill

    Process Payment

ELSE

    Display "No Cab Available"

ENDIF

END
```

Flowchart

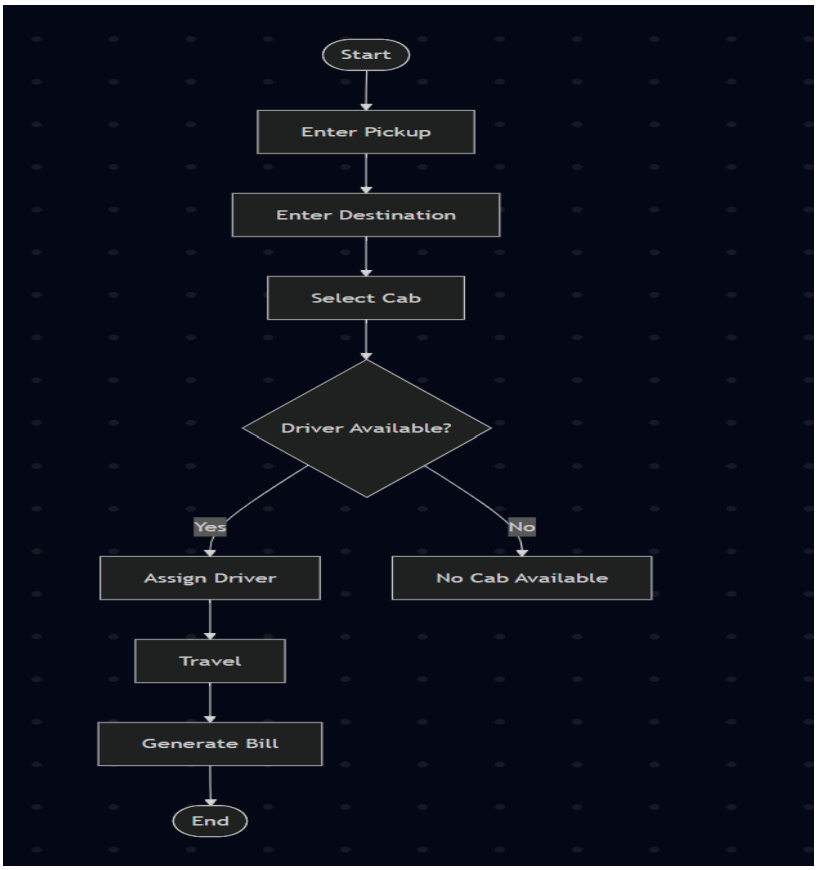


Figure 3: Flowchart of the OLA Cab Booking Process.

Advantages

- Quick and easy cab booking.
- Intelligent driver allocation.
- Real-time GPS tracking.
- Accurate fare estimation.
- Reduced waiting time.
- Safe and secure travel.
- Cashless payment facility.

Applications

- Ride-hailing services (OLA, Uber, Lyft).
- Taxi management systems.
- Logistics and delivery services.
- Fleet management.
- Smart transportation systems.
- Autonomous vehicle research.

Conclusion

The **OLA Cab Booking System** is an excellent real-world application of **Artificial Intelligence** that uses a **Goal-Based Problem-Solving Agent** to achieve efficient transportation. The agent receives customer requests, analyzes available resources, selects the most suitable cab, calculates fare and travel time, assigns the nearest driver, and ensures safe

navigation to the destination. By using GPS, traffic analysis, and intelligent decision-making, the OLA AI agent provides a reliable, cost-effective, and user-friendly transportation service. This demonstrates how AI agents can solve real-world problems through planning, reasoning, and goal-oriented decision-making.

Question 5: Solve the Logistics Delivery Network Using Uniform Cost Search (UCS)

Aim:

To determine the **least-cost path** from the starting warehouse **S** to the destination warehouse **G** using the **Uniform Cost Search (UCS)** algorithm.

Introduction:

Uniform Cost Search (UCS) is an **uninformed search algorithm** used in Artificial Intelligence to find the **minimum-cost path** between two nodes in a weighted graph. Unlike Breadth-First Search (BFS), UCS considers the **cost associated with each edge** instead of the number of edges.

In a logistics delivery network, every transportation route has an associated travel cost consisting of **fuel consumption, travel time, toll charges, and operational expenses**. The objective of UCS is to identify the most economical path that minimizes the overall transportation cost.

Since all edge costs are positive, UCS always expands the node with the **lowest cumulative cost**, guaranteeing an **optimal solution**.

Problem Statement:

A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost.

The delivery network is represented as a weighted graph.

The objective is to determine the **least-cost path** from the starting warehouse **S** to the destination warehouse **G** using the **Uniform Cost Search (UCS)** algorithm.

Input:

Warehouses (Nodes)

S, A, B, C, D, G

Routes (Edges with Cost)

Route	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

Delivery Network

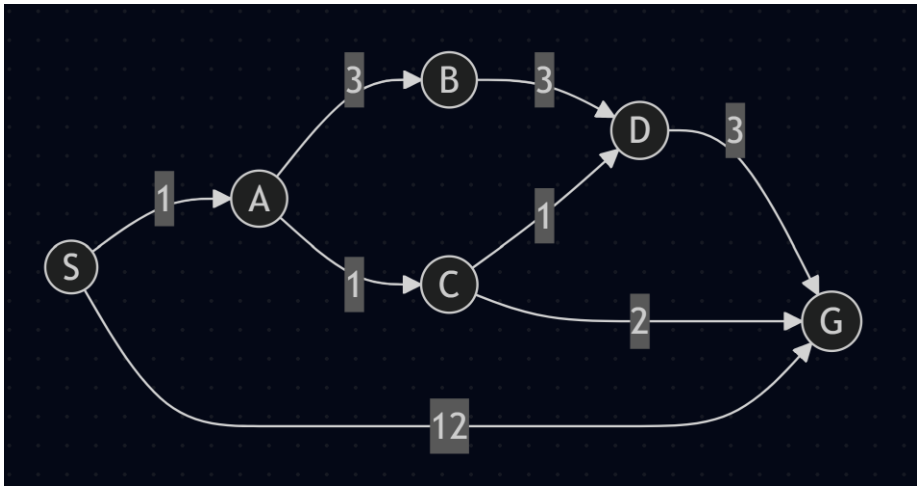


Figure 1: Weighted graph representing the logistics delivery network.

Algorithm Used:

Uniform Cost Search (UCS)
Algorithm Steps
Step 1: Insert the start node **S** into the priority queue with cost **0**.
Step 2: Remove the node having the minimum cumulative cost.
Step 3: Expand that node and generate all successor nodes.
Step 4: Calculate cumulative path costs.
Step 5: Insert unexplored nodes into the priority queue.
Step 6: Repeat until the goal node **G** is removed from the priority queue.

Initial State

Node	Cost
S	0

Priority Queue
[S : 0]

Goal State
Reach Warehouse G

Step-by-Step UCS Execution:

Iteration 1
Expand
S (Cost = 0)
Generated Nodes

Path	Cost
S → A	1
S → G	12

Priority Queue
A(1)

G(12)

Iteration 2
Expand
A (Cost = 1)
Generated Nodes

Path	Total Cost
S → A → B	4
S → A → C	2

Priority Queue
C(2)

B(4)

G(12)

Iteration 3
Expand
C (Cost = 2)
Generated Nodes

Path	Total Cost
S → A → C → D	3
S → A → C → G	4

Priority Queue
D(3)

B(4)

G(4)

G(12)

Iteration 4
Expand
D (Cost = 3)
Generated Node

Path	Total Cost
S → A → C → D → G	6

Priority Queue
B(4)

G(4)

G(6)

G(12)

Iteration 5
Expand
B (Cost = 4)
Generated Node

Path	Total Cost
S → A → B → D	7

Priority Queue
G(4)

G(6)

D(7)

G(12)

Iteration 6
Expand
G (Cost = 4)
Goal Found.
Algorithm Stops.

Search Tree

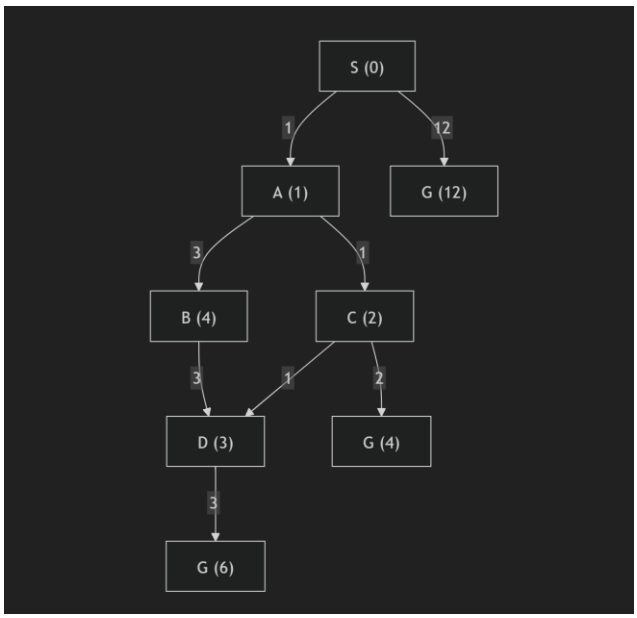


Figure 2: Uniform Cost Search Tree.

Priority Queue at Each Step

Step	Priority Queue
1	A(1), G(12)
2	C(2), B(4), G(12)
3	D(3), B(4), G(4), G(12)
4	B(4), G(4), G(6), D(7), G(12)
5	G(4), G(6), D(7), G(12)

Least Cost Path;
S → A → C → G
Total Cost
S → A = 1

A → C = 1

C → G = 2

Total Cost = 4

Result:

Path	Total Cost
S → A → C → G	**4**

Therefore,

The least-cost path from warehouse S to warehouse G using Uniform Cost Search (UCS) is:

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

with a **minimum total cost of 4**.

Advantages of Uniform Cost Search:

- Guarantees the optimal solution.
- Suitable for weighted graphs.
- Expands the least-cost node first.
- Finds the minimum transportation cost.
- Efficient for logistics and route optimization problems.

Applications:

- Logistics and supply chain optimization.
- GPS navigation systems.
- Route planning.
- Network routing.
- Robot path planning.
- Airline scheduling.
- Transportation management.

Conclusion:

Uniform Cost Search (UCS) is an optimal search algorithm that expands nodes in order of their cumulative path cost, ensuring that the least-cost solution is always found when all edge costs are positive. In this logistics delivery network, UCS systematically explored the available routes and determined that the path **S → A → C → G** has the minimum total transportation cost of **4**. This demonstrates the effectiveness of UCS in solving real-world optimization problems such as logistics, navigation, and route planning.

\